

## Programmation Fonctionnelle

### 18/10/2025 : Examen Partiel

Cet énoncé contient 4 pages. Lisez attentivement l'énoncé dans son intégralité avant de commencer à rédiger vos réponses. Tout document personnel en papier est autorisé. Les ordinateurs et les téléphones portables doivent être éteints et rangés, ainsi que tout autre moyen de communication.

Les fonctions demandées doivent être rédigées en style fonctionnel pur : ni références, ni tableaux, ni boucles `for` ou `while`, ni champs mutables. Chaque question peut utiliser les fonctions prédéfinies et/ou les fonctions des questions précédentes.

**Le sujet comporte trois exercices : écrivez chacun sur une feuille distincte afin de faciliter une correction plus rapide et efficace.**

**Exercice 1 :** Dans cet exercice, il est autorisé (mais pas obligatoire) d'utiliser les fonctions du module `List` de la bibliothèque standard d'OCaml.

**Bulletin de notes.** Nous allons représenter un bulletin de notes à l'aide des types OCaml suivants :

```
type resultat = Note of int | Absence of bool
type bulletin = resultat list
```

Ainsi, un résultat pour une épreuve donnée peut être soit une note (qui est un nombre entier), soit une absence qui peut être justifiée (avec la valeur `true`) ou non-justifiée (avec la valeur `false`). Un bulletin de notes est une liste de résultats.

Par exemple, la ligne suivante définit une valeur de type `bulletin` :

```
let exemple = [Note 8; Absence true; Note 14; Note 16; Absence false; Absence true]
```

1. Écrire une fonction

```
separer : bulletin -> int list * int * int
```

qui prend en entrée un bulletin, et renvoie un triplet constitué de :

- la liste des notes,
- le nombre d'absences justifiées,
- le nombre d'absences non-justifiées.

Ainsi, pour l'exemple donné, on doit avoir `separer exemple = ([8; 14; 16], 2, 1)`.

2. Écrire une fonction

```
verifier_notes : bulletin -> bool
```

qui vérifie que toutes les notes sont comprises entre 0 et 20 (inclus).

3. Écrire une fonction

```
moyenne : bulletin -> float
```

qui calcule la moyenne du bulletin, en prenant en compte que :

- les absences justifiées sont ignorées,
- les absences non-justifiées comptent comme des 0,
- et toutes les notes ont le même coefficient.

Ainsi, pour l'exemple donné, on doit avoir `moyenne exemple = 9.5`.

**Généralisation : listes et types sommes.** Le type `bulletin` des questions précédentes est un exemple de liste *hétérogène* : elle peut contenir des valeurs qui sont soit des entiers, soit des booléens. On peut généraliser cette idée à l'aide du type somme défini ci-dessous.

```
type ('a, 'b) sum = Gauche of 'a | Droite of 'b
```

Une valeur de type `('a, 'b) sum` peut être soit de type `'a`, soit de type `'b`. Le constructeur (`Gauche` ou `Droite`) permet de savoir quel est le type de l'élément. Ainsi, le type `(int, bool) sum` est isomorphe au type `resultat` vu précédemment.

4. Écrire une fonction

```
convertir : bulletin -> (int, bool) sum list
```

qui transforme une valeur de type `bulletin` en une liste de valeurs de type `(int, bool) sum` contenant les mêmes données.

5. Écrire une fonction

```
filter_map : ('a -> 'c option) -> ('b -> 'c option) -> ('a, 'b) sum list -> 'c list
```

telle que `filter_map f g l` parcourt la liste `l`, dont les éléments sont de type `('a, 'b) sum`, de la manière suivante :

- si un élément est de type `'a`, on applique la fonction `f` ;
- si un élément est de type `'b`, on applique la fonction `g` ;
- dans les deux cas, si l'application de la fonction renvoie `None`, l'élément est ignoré ;
- sinon, lorsque l'application de `f` ou `g` renvoie `Some c`, l'élément `c` est ajouté au résultat.

Ainsi, le résultat final de `filter_map f g l` est une liste de type `'c list`.

Par exemple, l'appel ci-dessous :

```
let f n = if n mod 2 = 0 then Some (n / 2) else None in
let g s = Some (String.length s) in
let l = [Gauche 4; Gauche 7; Droite "Hello"; Gauche 24; Droite "World!"] in
filter_map f g l
```

doit renvoyer la liste `[2; 5; 12; 6]` (où l'élément `Gauche 7` a été ignoré).

6. Réécrire la fonction `moyenne : bulletin -> float` de la question 1.3, sans utiliser la récursivité, en vous servant des fonctions précédentes `convertir` et `filter_map`, de la fonction `List.length`, et de la fonction `List.fold_left (+) 0`.

**Rappel : Chaque exercice doit être rédigé sur une feuille distincte.**

**Exercice 2 :** On considère les arbres binaires "à deux couleurs" représentés en OCaml à l'aide du type suivant :

```
type br_tree = Nil | B of br_tree * br_tree | R of br_tree * br_tree
```

1. Écrire la fonction

```
count_B: br_tree -> int
```

qui compte les noeuds internes étiquetés par `B`.

Par exemple `count_B (B (R (Nil, B (Nil, Nil)), Nil))` vaut 2.

2. Écrire la fonction

```
all_R: br_tree -> bool
```

qui vérifie si tous les noeuds internes sont étiquetés par `R`.

Par exemple `all_R (R (R (Nil, R (Nil, Nil)), Nil))` vaut `true`.

3. On dit qu'une valeur de type `br_tree` est *correcte* si sur chaque chemin de la racine à une feuille il y a autant de `B` que de `R`. Par exemple

`B (R (Nil, Nil), R (R (B (Nil, Nil), B (Nil, Nil)), Nil))`

est correcte, et `B (Nil, R (Nil, Nil))` ne l'est pas.

Écrire la fonction `is_correct : br_tree -> bool` qui vérifie si un arbre est correct.

4. On considère l'itérateur suivant

```
let rec fold_br fNil fB fR t =
  match t with
  | Nil -> fNil
  | B (l,r) -> fB (fold_br fNil fB fR l) (fold_br fNil fB fR r)
  | R (l,r) -> fR (fold_br fNil fB fR l) (fold_br fNil fB fR r)
```

de type

`'a -> ('a -> 'a -> 'a) -> ('a -> 'a -> 'a) -> br_tree -> 'a`

Redéfinir les fonctions des points 1 et 2 en utilisant `fold_br`.

5. On considère la fonction suivante, de type `br_tree -> int list`

```
let aux = fold_br [0] (fun x y -> List.map (fun n -> n+1) (x@y))
  (fun x y -> List.map (fun n -> n-1) (x@y))
```

- Donner la valeur de l'appel `aux (B (R (Nil, Nil), B (Nil, Nil)))`
- Redéfinir la fonction du point 3 en utilisant `aux`.

**Rappel : Chaque exercice doit être rédigé sur une feuille distincte.**

**Exercice 3 :** Une **file de priorité** est une file où chaque élément est associé à une **priorité** (un nombre, un ordre, un poids, etc.). Dans une file, les éléments sortent dans l'ordre de leur arrivée (FIFO : *First In, First Out*). Dans une file de priorité, les éléments sortent **selon leur priorité** :

- l'élément avec la priorité la plus basse (ou la plus haute, selon la convention adoptée) est retiré en premier,
- l'ordre de retrait n'est donc pas nécessairement l'ordre d'arrivée.

Nous allons utiliser la convention suivante : l'élément de la file ayant la priorité *minimale* est prioritaire. Le but de cet exercice est d'implémenter les files de priorité en utilisant des arbres binaires. On considère le type :

```
type ('a, 'p) tree =
  | Empty
  | Node of ('a * 'p) * ('a, 'p) tree * ('a, 'p) tree
```

où `'a` est le type des éléments de la file et `'p` le type des priorités.

Une valeur `t` de type `('a, 'p) tree` est dite *bien formée* si, **pour tout sous-arbre** de `t` de la forme `Node ( (x,p), g, d )`, la priorité `p` est minimale parmi toutes les priorités présentes dans les sous-arbres `g` et `d`. En particulier, la racine d'un arbre bien formé possède toujours la priorité minimale de l'arbre.

Dans la suite de l'exercice, une file de priorité sera une valeur bien formée de type `('a, 'p) tree`.

1. Expliquer en quoi la structure `('a, 'p) tree` peut représenter une file de priorité.

2. Écrire une fonction `is_empty` de type `('a', 'p) tree -> bool` qui teste si une file de priorité est vide. Par exemple
 

```
is_empty (Node ((1,0), Empty, Empty)) = false
```
3. Écrire une fonction `push` de type `('a * 'p) -> ('a, 'p) tree -> ('a, 'p) tree`. qui insère un couple dans une file de priorité. L'arbre renvoyé par `push` doit être bien formé.
4. Écrire une fonction `peek` de type `('a, 'p) tree -> 'a` qui retourne l'élément le plus prioritaire de la file sans le retirer. Si la file est vide, la fonction arrête le calcul par `failwith "File vide"`.
5. Écrire une fonction `pop` de type `('a, 'p) tree -> ('a, 'p) tree` qui prend en argument une file `t`, retire son élément prioritaire et reconstruit un arbre bien formé à partir des deux sous-arbres de `t`. À nouveau, si la file est vide, la fonction arrête le calcul par `failwith "File vide"`.

Vous trouverez ci-dessous des exemples d'appels des fonctions `push` et `pop`.

#### Exemple.

```
# let one = push ('a',4) Empty;;
val one : (char, int) tree =
  Node (('a', 4), Empty, Empty)
# let two = push ('b',2) one;;
val two : (char, int) tree =
  Node (('b', 2), Node (('a', 4), Empty, Empty), Empty)
# let three = push ('c',1) two;;
val three : (char, int) tree =
  Node (('c', 1), Node (('b', 2), Empty, Empty), Node (('a', 4), Empty, Empty))
# pop three;;
- : (char, int) tree = Node (('b', 2), Empty, Node (('a', 4), Empty, Empty))
```